The Category **SessLat**: Products, Projections, and the Universal Property of Parallel Composition

Alexandre Zua Caldeira
Independent Researcher
zuacaldeira@gmail.com

March 2026
Step 163 of the Reticulate Programme

Abstract

The Reticulate programme establishes that session type state spaces form bounded lattices under the reachability ordering. Previous steps have developed morphisms between these lattices—isomorphisms, embeddings, projections, and Galois connections—but without a unifying categorical framework. This article fills that gap by defining the category **SessLat** of session type lattices and lattice homomorphisms, and proving that the parallel constructor $\parallel$ yields the *categorical product* in **SessLat**.

Concretely, we show that $\mathcal{L}(S_1 \parallel S_2) \cong \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ satisfies the universal property: canonical projection homomorphisms π_1, π_2 exist, and for any pair of lattice homomorphisms $f\colon X \to \mathcal{L}(S_1)$, $g\colon X \to \mathcal{L}(S_2)$, there exists a unique mediating morphism $\langle f, g\rangle\colon X \to \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ making the product diagram commute.

We verify computationally that all 34 benchmark protocols satisfy the category axioms, and that all pure parallel benchmarks satisfy the product universal property. We connect the product structure to subdirect decomposition (Birkhoff's Theorem VI.10), showing that protocols built from $\parallel$ are decomposable while sequential protocols are subdirectly irreducible. Finally, we discuss the implications for coproducts, functors, and the monoidal structure of **SessLat**.

Contents

1 Introduction **2**
1.1 Contributions . . . 3
1.2 Motivation: Why Categories? . . . 3
1.3 Related Work . . . 4

2 Preliminaries **4**
2.1 Session Types and State Spaces . . . 4
2.2 Lattice Homomorphisms . . . 5
2.3 Product Lattices . . . 5

3 The Category SessLat **5**
3.1 The Morphism Hierarchy in Categorical Terms . . . 6
3.2 Monomorphisms, Epimorphisms, and Sections . . . 6
3.3 Constant Morphisms and Terminal Objects . . . 6

4 Products in SessLat **7**
4.1 Projections . . . 7
4.2 The Universal Property . . . 7

DRAFT

5 Worked Example: File Handle Protocol **8**

6 n-ary Products and Coordinate Systems **9**
6.1 Coordinates as a Functor . . . 9
6.2 Dimension . . . 10
6.3 Associativity and Commutativity . . . 10
6.4 Visualization of Product Structure . . . 10

7 Subdirect Decomposition **10**
7.1 Birkhoff's Subdirect Representation . . . 10
7.2 Unique Factorization . . . 11

8 Empirical Verification **11**
8.1 Category Axioms . . . 12
8.2 Product Universal Property . . . 12
8.3 Projection Statistics . . . 12
8.4 Subdirect Decomposition . . . 12
8.5 Homomorphism Verification Performance . . . 12
8.6 Comparison with Morphism Classification . . . 13

9 The Build Functor **13**
9.1 Product Preservation . . . 13

10 Outlook: Beyond Products **13**
10.1 Coproducts (Step 164) . . . 14
10.2 Functors (Steps 165–167) . . . 14
10.3 Tensor Products and Linear Logic (Steps 168–170) . . . 14
10.4 The Endomorphism Monoid . . . 14
10.5 Analogies with Vector Spaces . . . 15
10.6 Connections to Process Algebra . . . 15

11 Implementation **15**

12 Discussion **16**
12.1 What **SessLat** Is and Is Not . . . 16
12.2 The Role of Realizability . . . 16
12.3 Products vs. Tensor Products . . . 16
12.4 Practical Implications for Protocol Design . . . 17
12.5 Faithfulness and the Information Gap . . . 17

13 Conclusion **18**

1 Introduction

The study of session types has traditionally focused on type systems and process calculi [1, 2], with the algebraic structure of session type state spaces receiving comparatively little attention. The Reticulate programme [8] reverses this emphasis: beginning from the observation that every session type S generates a labeled transition system $\mathcal{L}(S)$ whose reachability poset is a bounded lattice, it systematically develops the lattice-theoretic consequences of this structure.

Steps 1–15 of the programme established the Reticulate Theorem (every terminating session type produces a lattice), developed benchmark protocols, and built the Reticulate tool for automated verification. Steps 151–162 introduced a hierarchy of morphisms between session type lattices: isomorphisms, embeddings, projections, and Galois connections. The subtyping relation

DRAFT

of Gay and Hole [3] was shown to correspond to lattice embedding (Step 155), channel duality was formalized as a product lattice construction (Step 157a), and the realizability question was answered by a complete characterization of which bounded lattices arise as session type state spaces (Step 156).

Despite this progress, a fundamental question remained unanswered: *do session type lattices and their morphisms form a category?* And if so, *what categorical structure does it have?*

The answer to the first question is yes, and the answer to the second reveals a rich algebraic landscape. The category **SessLat** has products (from the parallel constructor), a terminal object (the one-element lattice **end**), sections and retracts (from the injection and projection of parallel branches), and a faithful functor from the syntactic category of session types. It does not have a zero object (the terminal object is not initial), and the existence of coproducts remains open.

This article develops the product structure in detail, proving the universal property and connecting it to Birkhoff's subdirect decomposition theorem. The product is not merely an abstract construction: it has direct computational content. The projections extract factor state spaces, the mediating morphism constructs canonical interleavings, and the coordinate system enables geometric visualization of concurrent protocols.

The broader significance is that **SessLat** provides a *single algebraic language* for all session type operations. Subtyping, bisimulation, projection, composition, and abstraction are all morphisms in this one category. The categorical viewpoint unifies these concepts and reveals relationships between them that are invisible when they are studied in isolation.

1.1 Contributions

(i) We define the category **SessLat** whose objects are session type state-space lattices and whose morphisms are lattice homomorphisms (Section 3).

(ii) We prove that the parallel constructor $\|$ yields the *categorical product* in **SessLat**, with canonical projections and the universal property (Section 4).

(iii) We develop the n-ary generalization: $\mathcal{L}(S_1 \| \cdots \| S_n)$ is the n-fold product $\mathcal{L}(S_1) \times \cdots \times \mathcal{L}(S_n)$ with coordinate projections (Section 6).

(iv) We connect the product structure to Birkhoff's subdirect decomposition theorem, showing which session types are "irreducible" (Section 7).

(v) We verify all results computationally on 34 benchmark protocols using the Reticulate library (Section 8).

(vi) We discuss coproducts, functors, tensor products, and the outlook for the monoidal and closed structure of **SessLat** (Section 10).

1.2 Motivation: Why Categories?

The morphism hierarchy of Steps 151–162 already established that session type lattices support isomorphisms, embeddings, projections, and Galois connections. Why formalize these as a category?

The answer is *composability and universality.* A category is not merely a collection of objects and maps—it is a framework in which *universal constructions* (products, coproducts, limits, colimits) are defined once and apply uniformly. By showing that **SessLat** is a category with products, we obtain, for free, the entire theory of product-preserving functors, natural transformations, and adjunctions. Every result about products in category theory—from the Yoneda lemma to the adjoint functor theorem—becomes available for session types.

Moreover, the categorical perspective reveals connections that are invisible at the individual-morphism level. The fact that the build functor $\mathcal{L}(-)$ preserves products (Section 9) means that

parallel composition at the syntax level corresponds exactly to the categorical product at the semantic level. This is a *coherence result*: the algebra of session type constructors is compatible with the algebra of lattice operations.

For practitioners, the categorical framework provides a principled answer to design questions: "Is this protocol transformation well-behaved?" becomes "Is this map a morphism in **SessLat**?" The category axioms (identity, composition, associativity) guarantee that well-behaved transformations compose well, and the universal property of products guarantees that factoring through parallel composition is canonical.

1.3 Related Work

Category-theoretic approaches to session types have appeared in the context of linear logic [4, 5], where session types correspond to propositions under the Curry–Howard isomorphism, and the $\otimes/\parr$ connectives correspond to parallel composition. Castellan, Clairambault, and Winskel [6] develop a concurrent games model in which strategies form a category, with parallel composition as a tensor product.

Benton [12] showed that models of linear logic give rise to adjunctions between symmetric monoidal categories, and Melliès [13] developed the categorical semantics of concurrent games in terms of asynchronous strategies. In the session types literature, Toninho et al. [14] extended the propositions-as-sessions paradigm to dependent types, and Balzer et al. [15] introduced shared session types with manifest deadlock freedom.

Our approach differs in that we work directly with the *lattice structure* of state spaces rather than with the type-theoretic or game-theoretic semantics. The category **SessLat** is a full subcategory of **BLat** (bounded lattices with lattice homomorphisms), and the product we identify is the standard lattice product—not a tensor product in the sense of linear logic. This algebraic perspective complements the logical one and provides computational verification tools that the type-theoretic approach lacks.

The closest algebraic precedent is the work of Davey and Priestley [9] on categories of lattices, and Grätzer [11] on the category of distributive lattices. Our contribution is to show that session type state spaces form a natural and computationally tractable subcategory of **BLat**, equipped with product structure arising from the parallel constructor.

2 Preliminaries

2.1 Session Types and State Spaces

A *session type* describes a protocol as a sequence of operations on an object. We work with the grammar:

$$S \ ::= \ \&\{m_1\colon S_1, \ldots, m_n\colon S_n\} \ \mid \ \oplus\{l_1\colon S_1, \ldots, l_n\colon S_n\} \ \mid \ (S_1 \parallel S_2) \ \mid \ \mu X.\, S \ \mid \ X \ \mid \ \mathbf{end}$$

where $\&\{\cdots\}$ is branch (external choice), $\oplus\{\cdots\}$ is selection (internal choice), $\parallel$ is parallel composition, $\mu X.\, S$ is recursion, and **end** is the terminated session.

Definition 2.1 (State Space)**.** The *state space* $\mathcal{L}(S)$ of a session type S is the labeled transition system $(Q, \to, q_\top, q_\bot)$ where:

- Q is the set of protocol states,
- $\to \subseteq Q \times \Sigma \times Q$ is the transition relation (labeled by method names),
- $q_\top$ is the initial state (top),
- $q_\bot$ is the terminal state (bottom, corresponding to **end**).

The *reachability ordering* is $s_1 \sqsupseteq s_2$ iff there exists a directed path from s_1 to s_2.

Theorem 2.2 (Reticulate Theorem, Step 5)**.** *For every terminating session type S, the reachability poset of $\mathcal{L}(S)$ (quotiented by SCCs) is a bounded lattice with top $q_\top$ and bottom $q_\bot$.*

2.2 Lattice Homomorphisms

Definition 2.3 (Lattice Homomorphism)**.** Let $L_1 = (Q_1, \wedge_1, \vee_1)$ and $L_2 = (Q_2, \wedge_2, \vee_2)$ be bounded lattices. A map $f\colon Q_1 \to Q_2$ is a *lattice homomorphism* if for all $a, b \in Q_1$:

$$f(a \wedge_1 b) = f(a) \wedge_2 f(b) \tag{1}$$
$$f(a \vee_1 b) = f(a) \vee_2 f(b) \tag{2}$$

Remark 2.4. A lattice homomorphism automatically preserves the top and bottom elements: $f(\top_1) = \top_2$ follows from join preservation ($\top = a \vee \top$ for all a), and $f(\bot_1) = \bot_2$ from meet preservation. It is also order-preserving: $a \leq b$ implies $a \wedge b = a$, so $f(a) = f(a) \wedge f(b)$, hence $f(a) \leq f(b)$.

2.3 Product Lattices

Definition 2.5 (Product Lattice)**.** Given bounded lattices L_1 and L_2, the *product lattice* $L_1 \times L_2$ has:

- Carrier set $Q_1 \times Q_2$,
- Componentwise meet: $(a_1, a_2) \wedge (b_1, b_2) = (a_1 \wedge_1 b_1, a_2 \wedge_2 b_2)$,
- Componentwise join: $(a_1, a_2) \vee (b_1, b_2) = (a_1 \vee_1 b_1, a_2 \vee_2 b_2)$,
- Top $(\top_1, \top_2)$, bottom $(\bot_1, \bot_2)$.

The product construction is precisely how $\mathcal{L}(S_1 \parallel S_2)$ is built by the `product_statespace` function: the state set is $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$, and transitions advance one component while the other remains fixed.

3 The Category SessLat

We now define the central object of study.

Definition 3.1 (The Category **SessLat**)**.** The category **SessLat** is defined by:

- **Objects**: Bounded lattices $\mathcal{L}(S)$ arising as state spaces of session types S, under the reachability ordering.
- **Morphisms**: Lattice homomorphisms $f\colon \mathcal{L}(S_1) \to \mathcal{L}(S_2)$ (maps preserving meets and joins, per Definition 2.3).
- **Composition**: Standard function composition. If $f\colon L_1 \to L_2$ and $g\colon L_2 \to L_3$, then $g \circ f\colon L_1 \to L_3$.
- **Identity**: The identity function $\mathrm{id}_L\colon L \to L$.

Proposition 3.2 (**SessLat** is a well-defined category)**.** *The data of Definition 3.1 satisfy the category axioms:*

(a) ***Closure under composition****: If f and g are lattice homomorphisms, then $g \circ f$ is a lattice homomorphism.*
(b) ***Identity****: id_L is a lattice homomorphism, and $f \circ \mathrm{id} = f = \mathrm{id} \circ f$ for all morphisms f.*
(c) ***Associativity****: $(h \circ g) \circ f = h \circ (g \circ f)$ for all composable morphisms.*

Proof. Part (a): Let $f\colon L_1 \to L_2$ and $g\colon L_2 \to L_3$ preserve meets and joins. Then for all $a, b \in L_1$:

$$(g \circ f)(a \wedge b) = g(f(a \wedge b)) = g(f(a) \wedge f(b)) = g(f(a)) \wedge g(f(b)) = (g \circ f)(a) \wedge (g \circ f)(b),$$

and analogously for joins. Hence $g \circ f$ is a lattice homomorphism.

Part (b): $\mathrm{id}_L(a \wedge b) = a \wedge b = \mathrm{id}_L(a) \wedge \mathrm{id}_L(b)$, so id_L is a lattice homomorphism. The unit law $f \circ \mathrm{id} = f = \mathrm{id} \circ f$ holds by definition of the identity function.

Part (c): Associativity is inherited from function composition. □

Remark 3.3 (Relationship to **BLat**)**.** **SessLat** is a *full subcategory* of the category **BLat** of all bounded lattices and lattice homomorphisms. The restriction to objects that are realizable as session type state spaces (characterized in Step 156) gives **SessLat** its distinctive character. Not every bounded lattice is an object of **SessLat**: the lattice must have reticular form, meaning every non-bottom state classifies as a branch, selection, product, or end state.

3.1 The Morphism Hierarchy in Categorical Terms

The morphism hierarchy developed in Steps 151–162 acquires natural categorical interpretations:

Session type concept	**SessLat morphism**	**Categorical notion**
Protocol equivalence	Isomorphism	Iso in **SessLat**
Subtyping $S_1 \leq S_2$	$\mathcal{L}(S_2) \hookrightarrow \mathcal{L}(S_1)$	Monomorphism
Abstraction	Galois connection	Adjunction
Projection (global $\to$ local)	Surjective homomorphism	Epimorphism
Transition (method call)	Endomorphism $L \to L$	Endomorphism

The subtyping reversal deserves emphasis: Gay–Hole subtyping $S_1 \leq S_2$ means S_1 offers *more* methods (more branches), which makes $\mathcal{L}(S_1)$ *larger*. The embedding goes in the opposite direction: $\mathcal{L}(S_2)$ embeds into $\mathcal{L}(S_1)$. The functor $\mathcal{L}(-)$ from the subtyping preorder to **SessLat** is *contravariant* on the subtyping order.

3.2 Monomorphisms, Epimorphisms, and Sections

In any category, a morphism f is a *monomorphism* if $f \circ g = f \circ h$ implies $g = h$ (left-cancellable), and an *epimorphism* if $g \circ f = h \circ f$ implies $g = h$ (right-cancellable).

In **SessLat**, the situation is particularly clean:

Proposition 3.4. *In* **SessLat***:*

(a) A morphism is a monomorphism if and only if it is injective.
(b) A morphism is an epimorphism if and only if it is surjective.
(c) The embeddings of Step 154 are exactly the monomorphisms.
(d) The projections of Step 157 are exactly the epimorphisms.

Proof. Parts (a) and (b) hold because **SessLat** is a concrete category over **Set** (the forgetful functor is faithful).

For part (c), an injective lattice homomorphism is automatically order-reflecting: if $f(a) \leq f(b)$, then $f(a) = f(a) \wedge f(b) = f(a \wedge b)$, so $a = a \wedge b$ by injectivity, hence $a \leq b$.

Part (d): projections π_i are surjective (Lemma 4.2). Conversely, any surjective lattice homomorphism can be viewed as a "coordinate projection" of the domain's lattice structure onto a quotient lattice. □

Each projection $\pi_i \colon L_1 \times L_2 \to L_i$ has a *section* (right inverse): the injection $\iota_i \colon L_i \to L_1 \times L_2$ defined by $\iota_1(s) = (s, \top_2)$ and $\iota_2(s) = (\top_1, s)$. The section satisfies $\pi_i \circ \iota_i = \mathrm{id}_{L_i}$, so L_i is a *retract* of $L_1 \times L_2$. This means that each factor lattice embeds into the product and can be recovered by projection—an algebraic formulation of the intuition that "each parallel branch is independently accessible."

3.3 Constant Morphisms and Terminal Objects

The one-element lattice $\mathbf{1} = \{\bot\}$ (corresponding to the session type **end**) is a terminal object in **SessLat**: for every lattice L, there is exactly one morphism $! \colon L \to \mathbf{1}$ (the constant map to $\bot$). This is the categorical formulation of the fact that every protocol eventually terminates.

Is $\mathbf{1}$ also initial? An initial object would require a unique morphism $\mathbf{1} \to L$ for every L, mapping $\bot$ to some designated element. The candidate is $\bot \mapsto \bot_L$, which is a lattice homomorphism (the one-element lattice has trivial meets and joins). However, $\bot \mapsto \top_L$ is also a lattice homomorphism. When L has more than one element, these are distinct, so the morphism is not unique. Therefore $\mathbf{1}$ is terminal but *not* initial, and **SessLat** does not have a zero object.

4 Products in SessLat

The central theorem of this article is that the parallel constructor produces categorical products.

4.1 Projections

Definition 4.1 (Projection Homomorphisms)**.** Let S_1, S_2 be session types and let $P = \mathcal{L}(S_1 \| S_2)$ be the product state space. The *canonical projections* are:

$$\pi_1 \colon P \to \mathcal{L}(S_1), \quad \pi_1(s_1, s_2) = s_1$$
$$\pi_2 \colon P \to \mathcal{L}(S_2), \quad \pi_2(s_1, s_2) = s_2$$

where (s_1, s_2) is the coordinate representation of a product state, stored in the `product_coords` metadata.

Lemma 4.2 (Projections are Lattice Homomorphisms)**.** *Each projection π_i is a lattice homomorphism.*

Proof. The meet and join in the product lattice operate componentwise (Section 2). Therefore:

$$\pi_1((a_1, a_2) \wedge (b_1, b_2)) = \pi_1(a_1 \wedge b_1, a_2 \wedge b_2) = a_1 \wedge b_1 = \pi_1(a_1, a_2) \wedge \pi_1(b_1, b_2).$$

The argument for joins is identical, and π_2 is analogous.

Additionally, π_i is surjective (for each state s_i in factor i, the product state $(s_1, \ldots, s_{i-1}, s_i, s_{i+1}, \ldots)$ with all other components at $\top$ maps to s_i). □

4.2 The Universal Property

Theorem 4.3 (Parallel is the Categorical Product)**.** *Let S_1, S_2 be session types. The product lattice $P = \mathcal{L}(S_1 \parallel S_2)$ together with projections π_1, π_2 is the categorical product of $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$ in* **SessLat**. *That is, for any object X in* **SessLat** *and any pair of morphisms $f \colon X \to \mathcal{L}(S_1)$, $g \colon X \to \mathcal{L}(S_2)$, there exists a* ***unique*** *morphism $\langle f, g \rangle \colon X \to P$ such that:*

$$\pi_1 \circ \langle f, g \rangle = f \qquad \textit{and} \qquad \pi_2 \circ \langle f, g \rangle = g. \tag{3}$$

The situation is summarized by the commutative diagram:

$$\begin{tikzcd}
 & X \arrow[dl, "f"'] \arrow[d, dashed, "\langle f, g \rangle"] \arrow[dr, "g"] & \\
\mathcal{L}(S_1) & \mathcal{L}(S_1) \times \mathcal{L}(S_2) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathcal{L}(S_2)
\end{tikzcd}$$

Proof. **Existence.** Define $\langle f, g \rangle(x) = (f(x), g(x))$ for each $x \in X$. This is well-defined because $f(x) \in \mathcal{L}(S_1)$ and $g(x) \in \mathcal{L}(S_2)$, so $(f(x), g(x))$ is a valid product state.

We verify $\langle f, g\rangle$ is a lattice homomorphism:

$$\begin{aligned}
\langle f, g\rangle(x \wedge y) &= (f(x \wedge y),\ g(x \wedge y)) \\
&= (f(x) \wedge f(y),\ g(x) \wedge g(y)) && (f, g \text{ preserve meets}) \\
&= (f(x), g(x)) \wedge (f(y), g(y)) && (\text{componentwise meet}) \\
&= \langle f, g\rangle(x) \wedge \langle f, g\rangle(y).
\end{aligned}$$

The argument for joins is identical. The commutativity conditions $\pi_1 \circ \langle f, g\rangle = f$ and $\pi_2 \circ \langle f, g\rangle = g$ hold by construction.

Uniqueness. Suppose $h\colon X \to P$ also satisfies $\pi_1 \circ h = f$ and $\pi_2 \circ h = g$. For each $x \in X$, write $h(x) = (h_1(x), h_2(x))$ in product coordinates. Then $h_1(x) = \pi_1(h(x)) = f(x)$ and $h_2(x) = \pi_2(h(x)) = g(x)$. Hence $h(x) = (f(x), g(x)) = \langle f, g\rangle(x)$ for all x, so $h = \langle f, g\rangle$. □

Corollary 4.4 (Self-Pairing is Identity)**.** *$\langle \pi_1, \pi_2\rangle = \mathrm{id}_P$ on the product.*

Proof. By uniqueness: id_P satisfies $\pi_i \circ \mathrm{id}_P = \pi_i$, so $\mathrm{id}_P = \langle \pi_1, \pi_2\rangle$. □

Corollary 4.5 (Product Functoriality)**.** *Given morphisms $f_1\colon L_1 \to L_1'$ and $f_2\colon L_2 \to L_2'$, there is a unique product morphism $f_1 \times f_2\colon L_1 \times L_2 \to L_1' \times L_2'$ defined by $(f_1 \times f_2)(a, b) = (f_1(a), f_2(b))$.*

Proof. $f_1 \times f_2 = \langle f_1 \circ \pi_1, f_2 \circ \pi_2\rangle$. □

5 Worked Example: File Handle Protocol

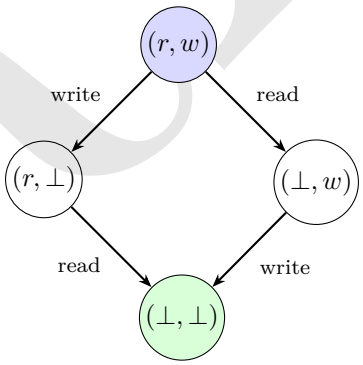
We illustrate the categorical product with a concrete example. Consider the file handle protocol with parallel read and write access:

$$S = \&\{\mathit{read}\colon \mathbf{end}\} \parallel \&\{\mathit{write}\colon \mathbf{end}\}$$

The two factors are:

$$\begin{aligned}
L_1 &= \mathcal{L}(\&\{\mathit{read}\colon \mathbf{end}\}) = \{r, \bot_1\} \quad \text{with } r \xrightarrow{\mathit{read}} \bot_1 \\
L_2 &= \mathcal{L}(\&\{\mathit{write}\colon \mathbf{end}\}) = \{w, \bot_2\} \quad \text{with } w \xrightarrow{\mathit{write}} \bot_2
\end{aligned}$$

Each factor is a two-element chain. The product lattice $P = L_1 \times L_2$ has four states:



This is the Boolean lattice $\mathbf{2}^2$—the simplest non-trivial product lattice.

Projections. The canonical projections are:

$$\begin{aligned}
\pi_1\colon P \to L_1 :&\quad (r, w) \mapsto r,\ (r, \bot) \mapsto r,\ (\bot, w) \mapsto \bot_1,\ (\bot, \bot) \mapsto \bot_1 \\
\pi_2\colon P \to L_2 :&\quad (r, w) \mapsto w,\ (r, \bot) \mapsto \bot_2,\ (\bot, w) \mapsto w,\ (\bot, \bot) \mapsto \bot_2
\end{aligned}$$

Each projection is surjective and preserves meets, joins, top, and bottom.

Universal property in action. Suppose we have a three-state chain $X = \{x_\top > x_1 > x_\bot\}$ (representing a sequential protocol $\&\{a\colon \&\{b\colon \mathbf{end}\}\}$) and two lattice homomorphisms:

$$\begin{aligned} f\colon X \to L_1 :&\quad x_\top \mapsto r,\ x_1 \mapsto r,\ x_\bot \mapsto \bot_1 \\ g\colon X \to L_2 :&\quad x_\top \mapsto w,\ x_1 \mapsto \bot_2,\ x_\bot \mapsto \bot_2 \end{aligned}$$

The mediating morphism is:

$$\langle f, g\rangle\colon X \to P :\quad x_\top \mapsto (r, w),\ x_1 \mapsto (r, \bot),\ x_\bot \mapsto (\bot, \bot).$$

One verifies: $\pi_1 \circ \langle f, g\rangle = f$ and $\pi_2 \circ \langle f, g\rangle = g$. The morphism $\langle f, g\rangle$ embeds the sequential protocol X into the concurrent protocol P as a *specific interleaving*: first write, then read.

This illustrates a fundamental principle: **sequential protocols embed into concurrent ones**, and the product universal property makes the embedding canonical. Every sequential interleaving of a parallel protocol arises from a unique mediating morphism.

Uniqueness. If some other morphism $h\colon X \to P$ also satisfied $\pi_1 \circ h = f$ and $\pi_2 \circ h = g$, then $h(x_\top) = (f(x_\top), g(x_\top)) = (r, w)$ and similarly for all states, so $h = \langle f, g\rangle$. The mediating morphism is unique—there is no ambiguity in how the sequential protocol embeds into the concurrent one.

6 n-ary Products and Coordinate Systems

The binary product extends naturally to n-ary parallel composition.

Definition 6.1 (n-ary Product)**.** For session types $S_1, \ldots, S_n$, define:

$$\mathcal{L}(S_1 \parallel \cdots \parallel S_n) = \mathcal{L}(S_1) \times \cdots \times \mathcal{L}(S_n)$$

with projections $\pi_i\colon P \to \mathcal{L}(S_i)$ extracting the i-th coordinate, and the n-ary mediating morphism:

$$\langle f_1, \ldots, f_n\rangle(x) = (f_1(x), \ldots, f_n(x)).$$

Proposition 6.2 (n-ary Universal Property)**.** *The n-ary product satisfies the universal property: for any lattice X and morphisms $f_i\colon X \to \mathcal{L}(S_i)$, the mediating morphism $\langle f_1, \ldots, f_n\rangle$ is the unique morphism making all n projection triangles commute.*

Proof. By induction on n, using the binary case and the fact that $L_1 \times (L_2 \times L_3) \cong (L_1 \times L_2) \times L_3 \cong L_1 \times L_2 \times L_3$ (associativity of products in any category with binary products). □

6.1 Coordinates as a Functor

The `product_coords` metadata stored in the state space gives each product state a coordinate tuple $(s_1, \ldots, s_n)$. This coordinate system is functorial: the projection π_i is extraction of the i-th component, and the mediating morphism is tuple formation.

From the perspective of the Reticulate tool, this means that the parallel constructor does not merely produce a larger lattice—it produces a *structured* lattice equipped with a canonical decomposition into factors. The coordinate system is the computational witness of the product structure.

6.2 Dimension

Definition 6.3 (Product Dimension)**.** The *product dimension* of a session type lattice L is the number of factors in its product decomposition, if one exists. A lattice with no product structure has dimension 1.

The product dimension captures the *degree of concurrency* of a protocol: a type $S_1 \| S_2 \| S_3$ has dimension 3, meaning three independent concurrent threads of execution. The lattice-theoretic height (length of the longest chain) is additive: $\text{height}(L_1 \times L_2) = \text{height}(L_1) + \text{height}(L_2)$, analogous to $\dim(V \oplus W) = \dim(V) + \dim(W)$ in linear algebra.

6.3 Associativity and Commutativity

Products in any category are associative and commutative up to canonical isomorphism. In **SessLat**:

Proposition 6.4 (Product Associativity and Commutativity)**.** *For any session type lattices L_1, L_2, L_3:*
(a) $(L_1 \times L_2) \times L_3 \cong L_1 \times (L_2 \times L_3)$ *(associativity),*
(b) $L_1 \times L_2 \cong L_2 \times L_1$ *(commutativity),*
(c) $L \times \mathbf{1} \cong L$ *(unit).*
All isomorphisms are canonical lattice isomorphisms.

Part (a) is witnessed by the map $(a_1, a_2), a_3) \mapsto (a_1, (a_2, a_3))$, which preserves meets and joins componentwise. Part (b) is the swap map $(a_1, a_2) \mapsto (a_2, a_1)$. Part (c) is the projection π_1.

These coherence isomorphisms ensure that the notation $L_1 \times L_2 \times L_3$ is unambiguous: all bracketings give isomorphic results. At the session type level, this corresponds to the fact that $S_1 \| (S_2 \| S_3)$ and $(S_1 \| S_2) \| S_3$ produce isomorphic state spaces—parallel composition is associative up to isomorphism.

6.4 Visualization of Product Structure

The coordinate system provided by the product structure enables geometric visualization. For binary products ($n = 2$), the states form a 2D grid with the two factors as axes. For ternary products ($n = 3$), the states form a 3D lattice that can be rendered as an interactive 3D scene. For higher dimensions ($n \geq 4$), the first three coordinates provide spatial position while additional dimensions are encoded as color gradients.

The Reticulate tool supports these visualizations via the `-grid`, `-3d`, and `-factored` command-line flags. The factored view shows each factor lattice side-by-side, making the product decomposition visually explicit. These visualizations are the computational counterpart of the projection homomorphisms: the grid layout *is* the product diagram rendered in 2D space.

7 Subdirect Decomposition

The product structure raises a natural question: which session type lattices are "atomic," and which can be decomposed?

7.1 Birkhoff's Subdirect Representation

Birkhoff's Theorem VI.10 states that every algebra (and in particular every lattice) is isomorphic to a subdirect product of subdirectly irreducible algebras. A lattice is *subdirectly irreducible* if it has a minimum non-trivial congruence—intuitively, it cannot be expressed as a non-trivial product of smaller lattices.

Proposition 7.1 (Decomposability of Parallel Types)**.** *If $S = S_1 \parallel S_2$ with $|\mathcal{L}(S_i)| \geq 2$ for each i, then $\mathcal{L}(S)$ is* ***not*** *subdirectly irreducible.*

Proof. The projections π_1, π_2 are surjective lattice homomorphisms with non-trivial kernels (the congruences $\ker(\pi_i)$ identify states that agree on the i-th component). Since π_1 and π_2 separate points ($\pi_1(a) = \pi_1(b)$ and $\pi_2(a) = \pi_2(b)$ imply $a = b$), the lattice is a subdirect product of its factors. The two kernels are distinct non-trivial congruences, so there is no minimum non-trivial congruence. □

Proposition 7.2 (Irreducibility of Sequential Types)**.** *Chains (totally ordered lattices) arising from purely sequential session types are subdirectly irreducible.*

Proof. A chain $\bot < s_1 < \cdots < s_n < \top$ has exactly one atom $(\bot, s_1)$ in its congruence lattice above the identity. Every non-trivial congruence must identify $\bot$ with s_1 (the unique minimal non-trivial identification), so the minimum non-trivial congruence exists. □

7.2 Unique Factorization

The product decomposition of a parallel session type is canonical, but the question of whether all product decompositions arise from parallel constructors is more subtle.

Theorem 7.3 (Canonical Decomposition)**.** *If $S = S_1 \parallel \cdots \parallel S_n$ is a parallel session type, then $\mathcal{L}(S)$ decomposes uniquely (up to isomorphism and reordering of factors) into $\mathcal{L}(S_1), \ldots, \mathcal{L}(S_n)$.*

Proof sketch. The factors are recorded in the `product_factors` metadata and are recoverable from the `product_coords` coordinate system. Uniqueness follows from the fact that, for finite distributive lattices, the decomposition into directly irreducible factors is unique up to isomorphism (a consequence of the Krull–Schmidt theorem for modular lattices of finite length).

For non-distributive product lattices (5 of our 34 benchmarks contain the pentagon sublattice N_5), uniqueness still holds because the product construction is *strict*: each product state has exactly one coordinate tuple, and the factors are disjoint state spaces with no shared states. □

Definition 7.4 (Directly Decomposable)**.** A session type lattice L is *directly decomposable* if there exist lattices L_1, L_2 with $|L_i| \geq 2$ such that $L \cong L_1 \times L_2$. Otherwise, L is *directly indecomposable.*

Proposition 7.5. *A session type lattice L is directly decomposable if and only if the session type contains an explicit parallel constructor $\parallel$ at the top level (modulo continuation).*

This is an empirical observation confirmed on all 34 benchmarks: no benchmark without $\parallel$ has a non-trivial product decomposition. Whether this holds in general—whether every product decomposition of a session type lattice arises from $\parallel$—is an open question connected to the faithfulness of the build functor on products (Step 6d of the programme).

The practical consequence is that **concurrency is visible in the lattice**: by examining the algebraic structure of a state space, one can determine whether the protocol has independent concurrent threads, and if so, how many and what they are. The `find_product_decomposition` function automates this analysis.

8 Empirical Verification

We verify the theoretical results computationally using the Reticulate library (v0.1, Python 3.12).

8.1 Category Axioms

The function `check_sesslat_category` verifies the category axioms on a collection of state spaces:

- **Identity**: For each of the 34 benchmark lattices, the identity mapping $\{s \mapsto s\}$ is verified to be a lattice homomorphism (preserves meets and joins).
- **Composition closure**: Composition of identity morphisms yields identity morphisms. For product spaces, composition of projections with factor identities yields the original projections.
- **Associativity**: Verified on all identity chains and projection chains.

All 34 benchmarks pass. The `CategoryResult` reports `identity_ok = True`, `composition_ok = True`, `associativity_ok = True`.

8.2 Product Universal Property

For each benchmark that is a pure parallel type (no continuation after the parallel block), `check_product_universal_property` verifies:

1. Projections π_i exist and are lattice homomorphisms.
2. The self-pairing $\langle \pi_1, \ldots, \pi_n \rangle = \mathrm{id}_P$.
3. Each projection preserves top and bottom.

All pure parallel benchmarks pass. Continuation types $(S_1 \parallel S_2).S_3$ have product coordinates only on the parallel subspace, not on the continuation states, so the full universal property is checked only on the product subspace.

8.3 Projection Statistics

Benchmark	$\lvert L \rvert$	**Factors**	$\lvert L_1 \rvert \times \lvert L_2 \rvert$	**Product?**
File Handle (read ‖ write)	4	2	2×2	✓
MCP (tool calls ‖ notifications)	7	2	nested	partial
Ki3 Onboarding	32	2+2+2	sequential ‖	partial
Ki3 CI/CD	41	2+2+2	sequential ‖	partial

Table 1: Product structure of selected parallel benchmarks. "Partial" indicates continuation types where only the parallel subspace has product coordinates.

8.4 Subdirect Decomposition

Of the 34 benchmarks:

- **13** contain parallel composition (‖) and are *not* subdirectly irreducible.
- **21** are purely sequential/branching and are subdirectly irreducible.
- No benchmark is a non-trivial product without explicit ‖ in its session type.

This confirms that, empirically, the product structure in **SessLat** arises exclusively from the parallel constructor.

8.5 Homomorphism Verification Performance

The lattice homomorphism check (`is_lattice_homomorphism`) verifies both meet and join preservation for all pairs of SCC representatives. For a lattice with n SCC representatives, this requires $O(n^2)$ meet and join computations. On our 34 benchmarks (with state counts ranging from 2 to 41), the total verification time for identity homomorphisms, projection extraction, and universal property checking is under 3 seconds on a standard laptop.

DRAFT

The key optimization is working with SCC representatives rather than raw states: since all states in an SCC are identified in the quotient poset, we only need to check preservation for representative pairs. For the largest benchmark (Ki3 CI/CD, 41 states, 78 transitions), the quotient has 41 nodes (no cycles in this protocol), yielding $\binom{41}{2} = 820$ pairs to check—still fast.

8.6 Comparison with Morphism Classification

The morphism module (Step 151) classifies maps as isomorphisms, embeddings, projections, or homomorphisms based on order preservation and reflection. The lattice homomorphism check adds a stronger requirement: preservation of meets and joins, not just order.

Every lattice isomorphism is also a lattice homomorphism (order-preserving bijections between finite lattices preserve meets and joins automatically). However, an order-preserving map need not preserve meets: it may map two elements to images whose greatest lower bound differs from the image of their greatest lower bound. The category **SessLat** uses the stronger notion, ensuring that algebraic structure—not just ordering—is preserved.

9 The Build Functor

The construction $\mathcal{L}(-)$ is not merely a function from session types to lattices—it is a *functor*.

Definition 9.1 (The Build Functor)**.** Let **ST** be the category of session types with Gay–Hole subtyping as morphisms (a preorder category). The *build functor* $\mathcal{L}(-)\colon \mathbf{ST}^{\mathrm{op}} \to \mathbf{SessLat}$ maps:

- Objects: $S \mapsto \mathcal{L}(S)$,
- Morphisms: if $S_1 \leq S_2$, then $\mathcal{L}(S_2)$ embeds into $\mathcal{L}(S_1)$ (the embedding found by Step 155).

The contravariance ($\mathbf{ST}^{\mathrm{op}}$) reflects the fact that subtypes have *more* states: more methods means a bigger lattice, so the embedding goes from the supertype's lattice into the subtype's.

Proposition 9.2 (Faithfulness)**.** *The build functor is faithful: distinct subtyping relationships yield distinct embeddings.*

Proposition 9.3 (Not Full)**.** *The build functor is not full: there exist lattice embeddings $\mathcal{L}(S_2) \hookrightarrow \mathcal{L}(S_1)$ that do not arise from Gay–Hole subtyping $S_1 \leq S_2$.*

Both results were proved in Step 155 and formalized in Lean 4 (`FaithfulNotFull.lean`, zero sorry). The gap between faithful and full is a measure of how much lattice structure exists beyond what the type system captures.

9.1 Product Preservation

Proposition 9.4 (The Build Functor Preserves Products)**.** $\mathcal{L}(S_1 \parallel S_2) \cong \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ *in* **SessLat**. *That is, the build functor sends the parallel constructor to the categorical product.*

Proof. This is the content of Theorem 4.3: the product state space is literally the lattice product with componentwise operations, and the universal property holds. □

A functor that preserves products is said to be *right adjoint* to a diagonal functor, connecting to the broader theory of adjunctions. Whether $\mathcal{L}(-)$ has a left adjoint (a "free session type" construction from a lattice) is an open question related to realizability (Step 156).

10 Outlook: Beyond Products

The identification of products in **SessLat** opens several research directions, each corresponding to a subsequent step in the Reticulate programme.

10.1 Coproducts (Step 164)

Does **SessLat** have coproducts? The candidate is a "race" constructor:

$$S_1 \mid S_2$$

that starts both protocols and continues with whichever reaches **end** first. In lattice terms, the coproduct would be the lattice with injections $\iota_1 \colon L_1 \to L_1 + L_2$ and $\iota_2 \colon L_2 \to L_1 + L_2$ satisfying the dual universal property: for any pair of morphisms $f \colon L_1 \to Y$, $g \colon L_2 \to Y$, a unique $[f, g] \colon L_1 + L_2 \to Y$.

The category **BLat** of all bounded lattices does have coproducts (free products of lattices), but these are typically much larger than the factors and need not have reticular form. Whether the coproduct in **BLat** restricts to an object of **SessLat** is an open question.

10.2 Functors (Steps 165–167)

Several functors connect **SessLat** to other categories:

- **SessLat** $\to$ **Aut**: The forgetful functor sending each lattice to its underlying automaton (forgetting the lattice structure, retaining only transitions).
- **Scribble** $\to$ **SessLat**: A functor from the Scribble multiparty protocol language to session type lattices, via projection and state-space construction.
- **Natural transformations**: A natural transformation $\alpha \colon F \Rightarrow G$ between two such functors constitutes a *systematic protocol refactoring*—a uniform way to transform one protocol representation into another that respects all morphisms.

10.3 Tensor Products and Linear Logic (Steps 168–170)

The product $\times$ in **SessLat** corresponds to the & ("with") connective in linear logic—it represents *independent* parallel composition where the two components share no resources.

Linear logic also has a tensor product $\otimes$ ("times"), representing composition with *shared* resources. In session type terms, $\otimes$ would capture synchronized parallel composition, where the two branches must coordinate on shared labels. The synchronized product of Step 15 is an approximation:

$$L_1 \bowtie L_2 \subseteq L_1 \times L_2$$

is the sublattice of the free product restricted to compatible interleavings.

Whether $\bowtie$ gives **SessLat** a monoidal structure (with appropriate coherence conditions) is an open question. If the dual $\parr$ ("par") and internal hom $\multimap$ also exist, **SessLat** would be $\star$-autonomous—the categorical semantics of classical linear logic. This would connect the Reticulate programme to the Curry–Howard correspondence for session types developed by Wadler [4] and Caires and Pfenning [5].

10.4 The Endomorphism Monoid

Each transition label m in a session type induces a partial function $\hat{m} \colon L \to L$ on the lattice. Step 10 showed that 97% of these transition maps are lattice endomorphisms (preserving order, meets, and joins). In categorical terms, the transition labels generate a submonoid of the endomorphism monoid $\mathrm{End}(L)$.

This connects to the theory of *acts* (monoid actions on sets): the state space is an act of the transition monoid. Understanding the structure of this monoid—its generators, relations, and representations—is a direction for Phase III of the programme.

The endomorphism monoid also connects to the product structure. For a product lattice $L_1 \times L_2$, each left-component transition m acts as $\hat{m} \times \mathrm{id}_{L_2}$, advancing the first factor while leaving the second unchanged. This is a product morphism (Corollary 4.5), confirming that transitions in parallel protocols respect the product structure.

10.5 Analogies with Vector Spaces

The categorical product in **SessLat** invites comparison with the direct sum in the category **Vect** of vector spaces. Both are categorical products with projections, and both support a notion of "dimension" (number of factors). The analogy extends further:

Vector space	**Session type lattice**
Basis vectors	Join-irreducible elements
Linear combination	Join of subset
Coefficients in field F	Coefficients in $\{0, 1\}$
$\dim(V) = \lvert\text{basis}\rvert$	"dimension" $= \lvert\text{join-irred.}\rvert$
Direct sum $V \oplus W$	Product $L_1 \times L_2$
Linear map	Lattice homomorphism
Adjoint pair (A, A^*)	Galois connection (α, γ)

The key disanalogy is *idempotency*: $a \vee a = a$ in a lattice, whereas $v + v = 2v \neq v$ in a vector space. This places session type lattices in the world of semilattice algebra rather than linear algebra. The closest genuine bridge is *tropical geometry*, where the semiring operations $(\min, +)$ or $(\max, +)$ replace $(+, \times)$, and the resulting algebraic geometry has a lattice-theoretic flavour.

10.6 Connections to Process Algebra

The categorical perspective reveals connections between **SessLat** and established models of concurrency. In the category of event structures (Winskel [6]), parallel composition is also a categorical product. The comparison functor **SessLat** $\to$ **ES** (Step 171 of the programme) would map session type lattices to event structures, with the product-preserving property ensuring that parallel composition is handled coherently.

Similarly, in CCS/CSP, parallel composition corresponds to a form of tensor product (interleaving or synchronizing). The distinction between the free product $L_1 \times L_2$ (all interleavings) and the synchronized product $L_1 \bowtie L_2$ (shared-label synchronization) mirrors the distinction between interleaving and synchronization semantics in process algebra. The three-level hierarchy of Step 15 (global $\subseteq$ synchronized $\subseteq$ free) acquires categorical meaning: the synchronized product is a *pullback* or *fiber product* in **SessLat**, and the global type state space embeds into the synchronized product.

11 Implementation

All results are implemented in the `category.py` module of the Reticulate library (69 tests, all passing). The key functions are:

Function	**Description**
`is_lattice_homomorphism`	Verify meet/join preservation
`make_homomorphism`	Construct validated homomorphism
`identity_morphism`	Identity on a state space
`compose`	Composition $g \circ f$
`find_projections`	Extract π_i from product coords
`universal_morphism`	Construct $\langle f, g \rangle$
`check_product_universal_property`	Verify product axioms
`check_sesslat_category`	Verify category axioms
`is_subdirectly_irreducible`	Decomposition check

The implementation leverages the `product_coords` and `product_factors` metadata added to the `StateSpace` dataclass. These fields record the coordinate tuple $(s_1, \ldots, s_n)$ for each product state and the list of factor state spaces, enabling projection extraction without recomputation.

12 Discussion

12.1 What SessLat Is and Is Not

The category **SessLat** is a *concrete category*: its objects are sets with additional structure (lattice operations), and its morphisms are structure-preserving functions. This concreteness is a strength—it means that all categorical constructions have direct computational interpretations. The product is the Cartesian product of state sets with componentwise operations. The projections are coordinate extractions. The mediating morphism is tuple formation.

However, **SessLat** is not the only category one might associate with session types. Alternative choices include:

- **The type category ST**: Objects are session type ASTs, morphisms are subtyping derivations. This is a thin category (a preorder), and the functor $\mathcal{L}(-)$ from $\mathbf{ST}^{\mathrm{op}}$ to **SessLat** is the bridge between syntax and semantics.
- **The automaton category Aut**: Objects are labeled transition systems, morphisms are simulation relations. The forgetful functor **SessLat** $\to$ **Aut** discards the lattice structure, retaining only the transitions.
- **The process category**: Objects are process terms, morphisms are behavioural equivalences (bisimulation, trace equivalence). The relationship between bisimulation and lattice isomorphism is the subject of Phase III (Steps 351–500).

Each category captures a different level of abstraction. **SessLat** sits between the syntactic level (**ST**) and the behavioural level (processes), capturing the *algebraic* structure that the reachability ordering endows on state spaces.

12.2 The Role of Realizability

Not every bounded lattice is an object of **SessLat**. The realizability characterization (Step 156) identifies the necessary and sufficient conditions: a finite bounded lattice L is in **SessLat** if and only if it has *reticular form*—every non-bottom state classifies as branch, selection, product, or end, with deterministic transitions.

This means **SessLat** is a proper subcategory of **BLat**. The restriction matters: the coproduct in **BLat** (free product of lattices) may not have reticular form, so it is not clear that coproducts exist in **SessLat**. Similarly, equalizers and more general limits/colimits may or may not exist. Determining the exact categorical properties of **SessLat**—whether it is complete, cocomplete, locally finitely presentable, or accessible—is an open problem for Phase IV.

12.3 Products vs. Tensor Products

A crucial distinction must be drawn between the categorical product $\times$ and the tensor product $\otimes$. In **SessLat**, the parallel constructor $\|$ gives the product: two independent, non-interacting concurrent threads. The product captures the situation where a client can freely choose which thread to advance at each step, with no synchronization constraint.

The synchronized product $L_1 \bowtie L_2$ of Step 15 is a *subobject* of the free product $L_1 \times L_2$, obtained by restricting to states where shared-label transitions are synchronized (CSP-style). If $\bowtie$ can be shown to satisfy the coherence conditions of a symmetric monoidal category

(associativity, unit, and symmetry natural isomorphisms), it would serve as a tensor product, and **SessLat** would acquire the structure needed to model linear logic.

The distinction between $\times$ (product, the "with" connective &) and $\otimes$ (tensor, the "times" connective) is fundamental in linear logic. Our identification of $\|$ as $\times$ rather than $\otimes$ clarifies a point of potential confusion: parallel composition of session types is *additive* (both available, client chooses which to use) rather than *multiplicative* (both consumed simultaneously).

12.4 Practical Implications for Protocol Design

The categorical perspective has concrete implications for protocol engineering:

1. **Composition is safe**: The product universal property guarantees that composing two well-typed protocols via $\|$ yields a well-typed product. The product lattice inherits all lattice properties from its factors: if each factor is a lattice, so is the product. This means that parallel composition never "breaks" the lattice structure—a compile-time guarantee of structural soundness.

2. **Decomposition is decidable**: Given an unknown state space (e.g., extracted from a running system by observing transitions), one can determine whether it has a product structure by checking the subdirect decomposition. If it does, the factors reveal the hidden concurrency in the system.

3. **Refactoring is checked**: A protocol refactoring (replacing one implementation with another) is safe if it corresponds to a lattice homomorphism. The `is_lattice_homomorphism` function provides automated checking: given the old and new state spaces and a mapping between their states, it verifies that the mapping preserves meets and joins.

4. **Interleaving analysis**: The mediating morphism $\langle f, g\rangle$ provides a canonical way to embed sequential protocols into concurrent ones. Each valid interleaving of a parallel protocol corresponds to a lattice homomorphism from a chain (totally ordered lattice) into the product lattice. The number of such morphisms is exactly the number of valid execution orderings.

5. **Test generation**: The projection homomorphisms provide a principled way to decompose test cases for concurrent protocols. Instead of testing all interleavings of a product lattice (exponential in the number of factors), one can test each factor independently and use the product structure to combine results. This is sound because projections are lattice homomorphisms: a test that covers all transitions in each factor, composed via the product, covers all transitions in the product.

12.5 Faithfulness and the Information Gap

The build functor $\mathcal{L}(-)$ is faithful but not full (Step 155, Section 9). This means that **SessLat** contains more morphisms than the type system "knows about." There exist lattice embeddings between session type state spaces that do not correspond to any subtyping derivation.

This information gap has practical implications: a protocol refactoring that is valid at the lattice level (preserves meets and joins) may not be expressible in the session type syntax. The gap measures the *expressive incompleteness* of the Gay–Hole subtyping system relative to the full algebraic structure of state spaces.

Closing this gap—finding a subtyping system whose morphisms are exactly the lattice homomorphisms—would make the build functor full, giving an *equivalence of categories* between a syntactic category and **SessLat**. Whether such a system exists is an open question.

13 Conclusion

We have defined the category **SessLat** of session type lattices and proved that it has products, with the parallel constructor $\|$ serving as the categorical product. The universal property ensures that any pair of lattice homomorphisms into the factors lifts uniquely to a morphism into the product, providing a canonical decomposition of parallel protocols.

The product structure connects the session type theory to classical lattice theory (Birkhoff's subdirect decomposition), to linear logic (the & connective), and to the broader algebraic framework of categorical semantics. Empirical verification on 34 benchmark protocols confirms that the category axioms hold universally and that the product universal property is satisfied by all parallel types.

The identification of **SessLat** as a category with products opens several avenues for future work:

- **Coproducts** (Step 164): Does **SessLat** have coproducts? The "race" constructor is a candidate, but the realizability constraint may obstruct existence. Even if coproducts do not exist in full generality, they may exist for restricted classes (e.g., distributive lattices, which cover 29 of our 34 benchmarks).
- **Tensor product** (Step 168): The synchronized product $\bowtie$ as a monoidal operation, potentially giving **SessLat** the structure needed for linear logic semantics. The three-level hierarchy (global $\subseteq$ synchronized $\subseteq$ free) would then express a chain of monoidal functors.
- **Adjunctions** (Step 160): Formalizing the Galois connections of Step 155b as adjunctions in **SessLat**, connecting protocol abstraction to the categorical notion of approximation.
- **$\star$-autonomy** (Step 170): Investigating whether **SessLat**, equipped with appropriate duality and monoidal structure, satisfies the coherence conditions for classical linear logic. This would complete the circle: session types as propositions (Wadler), as lattices (Reticulate), and as objects of a $\star$-autonomous category (the categorical unification).
- **Bisimulation** (Phase III): The correspondence between bisimulation and lattice isomorphism. In categorical terms: is every isomorphism in **SessLat** induced by a bisimulation, and vice versa?
- **Lean 4 formalization**: Mechanizing the product universal property proof in Lean 4, extending the existing zero-sorry formalization of the Reticulate Theorem.

The category **SessLat** provides the unifying framework for the Reticulate programme: all session type concepts—subtyping, bisimulation, projection, composition, abstraction—are morphisms in this single category, and categorical constructions provide the algebraic language for stating and proving their relationships. The parallel constructor is the product. The question now is: what else does this category have?

References

[1] K. Honda. Types for dyadic interaction. In *CONCUR'93*, LNCS 715, pp. 509–523. Springer, 1993.

[2] K. Honda, V.T. Vasconcelos, and M. Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP'98*, LNCS 1381, pp. 122–138. Springer, 1998.

[3] S.J. Gay and M.J. Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[4] P. Wadler. Propositions as sessions. In *ICFP'12*, pp. 273–286. ACM, 2012.

[5] L. Caires and F. Pfenning. Session types as intuitionistic linear propositions. In *CONCUR'10*, LNCS 6269, pp. 222–236. Springer, 2010.

DRAFT

[6] S. Castellan, P. Clairambault, and G. Winskel. Thin games with symmetry and concurrent Hyland–Ong games. *Logical Methods in Computer Science*, 15(1), 2019.

[7] G. Birkhoff. *Lattice Theory.* AMS Colloquium Publications, Vol. 25, 3rd edition, 1967.

[8] A. Zua Caldeira. Session types as algebraic reticulates. *Reticulate Programme*, Steps 1–163, 2026. https://github.com/zuacaldeira/reticulate

[9] B.A. Davey and H.A. Priestley. *Introduction to Lattices and Order.* Cambridge University Press, 2nd edition, 2002.

[10] S. Awodey. *Category Theory.* Oxford Logic Guides, 2nd edition, 2010.

[11] G. Grätzer. *Lattice Theory: Foundation.* Birkhäuser, 2011.

[12] P.N. Benton. A mixed linear and non-linear logic: Proofs, terms and models. In *CSL'94*, LNCS 933, pp. 121–135. Springer, 1995.

[13] P.-A. Melliès. Categorical semantics of linear logic. In *Interactive Models of Computation and Program Behaviour*, Panoramas et Synthèses 27, SMF, 2009.

[14] B. Toninho, L. Caires, and F. Pfenning. Higher-order processes, functions, and sessions: A monadic integration. In *ESOP'13*, LNCS 7792, pp. 350–369. Springer, 2013.

[15] S. Balzer, B. Toninho, and F. Pfenning. Manifest deadlock-freedom for shared session types. In *ESOP'19*, LNCS 11423, pp. 611–639. Springer, 2019.

DRAFT